Day 5 — Models and Inference

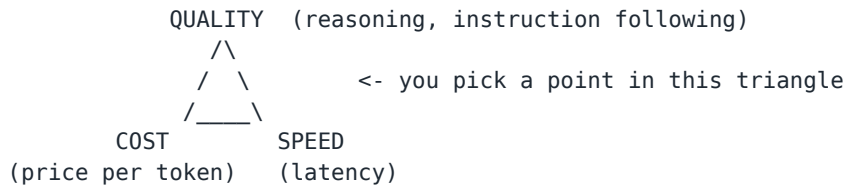
Sunday 25 Oct 2026 · AWS AIP-C01 exam tasks: **1.2 Select and configure models · 2.4 Call model APIs · 2.2 Deploy models** · Exam: Mon 9 Nov, 1:30 PM IST

Today in one paragraph
Choose the **smallest model that is good enough** (prove it with a model evaluation), configure how it writes with inference parameters, call it through the **Converse API** (streaming for responsiveness, an SQS queue for long or bursty work, retries with backoff for throttling, cross-region inference for extra capacity), and pay for it the right way: **on-demand** by default, **Provisioned Throughput** for steady guaranteed capacity, **batch** for large non-urgent jobs. Customise only when prompting and RAG are not enough.

Part 1 — Choosing and configuring a foundation model (Task 1.2)

1.1 Why model choice is a real decision

A **foundation model (FM)** is a large, general-purpose AI model trained on huge amounts of data, which you use without training it yourself. Amazon Bedrock offers many through one API: Amazon Nova, Anthropic Claude, Meta Llama, Mistral, Cohere and others. There is no single best model — every model trades off three things:



Larger models are usually smarter but slower and pricier; smaller models are fast and cheap but weaker on hard reasoning. **Example:** classifying 5 million product reviews (simple task, huge volume) → a small, cheap model; drafting legal responses (costly mistakes, low volume) → a large, capable model.

Exam phrasing
'The MOST cost-effective model that meets the quality requirement' → the **smallest model that is good enough**, never automatically the most powerful one.

1.2 Selection criteria

1. **Task quality** — can it actually do the job (reasoning, summarising, coding, classifying)?
2. **Modality** — the type of data handled: text-only (Nova Micro); **multimodal** input such as images, documents, video (Nova Lite, Nova Pro, Claude); image generation (Nova Canvas); video generation (Nova Reel); **embeddings** for search (Titan Text Embeddings V2, Cohere Embed). Example: photos of damaged parcels → multimodal model.
3. **Context window** — maximum tokens per request (a token is roughly ¾ of a word). A 300-page contract in one pass needs a large window.
4. **Latency** — live chat needs fast responses; an overnight report doesn't.
5. **Cost** — charged per input and output token; output tokens usually cost more.
6. **Features** — tool use, streaming, vision, customisation support.
7. **Language support** — e.g. Hindi or Japanese performance.
8. **Region availability / data residency** — the model must exist where your data may live (Day 4).

1.3 Choose with evidence: Amazon Bedrock model evaluation

Run your own prompt dataset through several candidate models and score them:

```
your 200 real prompts --> Model A --> answers --+
                      --> Model B --> answers --+--> scores + cost/latency --> pick the winner
                      --> Model C --> answers --+
```

Evaluation type	How it scores	Use when
Automatic	Built-in metrics (accuracy, robustness) against reference answers	You have correct answers to compare against
LLM-as-a-judge	A strong model grades each answer (helpfulness, correctness...)	Open-ended answers, no single right answer
Human	Your own reviewers rate outputs	Subjective quality (brand tone) or high stakes

1.4 Inference parameters — how the model writes

Inference means running a model to get output. A model writes **one token at a time**: at each step it computes a probability for every possible next token, then picks one. Parameters change how that pick is made.

```
Prompt: "The weather today is ..."
next-token probabilities:  sunny 50% | cloudy 25% | rainy 15% | purple 0.1% | ...
```

- **Temperature** (usually 0–1) — randomness of the pick. Low → consistent, predictable (invoice-number extraction: 0). High → varied, creative, more error-prone (five slogan ideas: ~0.8).
- **Top-p (nucleus sampling)** — consider only the smallest set of top tokens whose probabilities **add up to p**. It is a probability threshold, so the number of candidate tokens changes every step (p = 0.9: sunny+cloudy+rainy = 90% → 3 candidates; next step 'Paris' 95% → 1 candidate).
- **Top-k** — consider only the **k** most likely tokens (a fixed count). Not every model supports it.
- **Max tokens** — maximum answer length. When reached, generation stops there and the stop reason is **max_tokens** (answers cut off mid-sentence → raise it).
- **Stop sequences** — strings that end generation when produced (e.g. a blank line).
- **System prompt** — the model's role and rules.

Symptom or goal	Adjust
Answers vary every time but must be consistent	Lower temperature (and/or top-p)
Creative output too repetitive	Raise temperature
Answers cut off mid-sentence (stop reason max_tokens)	Raise max tokens
Model keeps writing past what you need	Stop sequence or lower max tokens
Costs high because answers are long	Lower max tokens; ask for concise answers

1.5 Your question: why isn't hallucination a temperature problem?

High temperature does raise the risk of made-up content, but it is not the root cause. Hallucinations have two causes:

Cause 1 — the model doesn't know the fact (main cause). Its most likely answer is already wrong:

```
Question: "What is AcmeShop's refund window?"   (real answer: 14 days; model never saw the policy)
   "30" 60%  <- most common refund window on the internet
   "14" 10%   "7" 8%   ...
Temperature 0 -> always "30": wrong, every time, confidently
Temperature 1 -> sometimes 30, 7 or 14: wrong in different ways
```

Cause 2 — randomness (secondary). The model knows the answer but high temperature picks an unlikely wrong token; lowering temperature fixes this case.

Grounding (RAG plus the Guardrails contextual grounding check) puts the facts in the prompt, which changes the probabilities themselves:

```
Prompt contains: "AcmeShop policy: refunds accepted within 14 days of delivery."
   "14" 97%   "30" 1%   <- the correct token is now the most likely
```

Scenario wording	Real cause	Fix
States facts not in our documents	Missing knowledge	Grounding (RAG + contextual grounding check)
Different answer each time for the same question	Randomness	Lower temperature / top-p
Both made-up and inconsistent	Both	Grounding + low temperature

One-line rule
Temperature controls **consistency**; grounding controls **correctness**.

Part 2 — How your application calls the model (Task 2.4)

2.1 InvokeModel vs Converse

InvokeModel is the older, model-specific operation: the JSON body must match each provider's own format, so switching models means rewriting request and response handling. **Converse** is the unified operation: one format for messages, system prompt, inference settings (**inferenceConfig**) and tools (**toolConfig**) across supported models — switching models changes only the model ID.

```
brt.converse(modelId="amazon.nova-lite-v1:0", messages=[...], inferenceConfig={"maxTokens": 500, "temperatu
brt.converse(modelId="<a Claude model ID>",    messages=[...], inferenceConfig={"maxTokens": 500, "temperatu
#            ^ only this changes
```

Example: Model B turns out 40% cheaper with equal quality — with Converse the switch is one line.

2.2 Streaming

Without streaming the app waits for the whole answer; with **ConverseStream** (or InvokeModelWithResponseStream) tokens arrive as they are generated.

```
WITHOUT streaming: [........ 15 s waiting ........] -> full answer appears
WITH streaming:    [0.4 s] The -> refund -> policy -> allows -> ...
                    ^ time to first token (TTFT): what the user feels
```

Streaming improves **perceived latency** only. It doesn't finish the full answer sooner and doesn't reduce cost — same tokens, same price.

2.3 Synchronous vs asynchronous

A synchronous call waits for the answer in the same request. Every link has a timeout: **Amazon API Gateway** (managed front door for HTTP APIs) defaults to **29 seconds**; **AWS Lambda** runs at most **15 minutes**. A 2-minute report through a default synchronous API fails. Fix: accept now, deliver later.

```
1. Browser -> API Gateway -> Lambda: enqueue job -> returns "Job 123 accepted"
                                  |
                           Amazon SQS queue   (SQS = Simple Queue Service)
2.                         Worker Lambda -> Bedrock (2 minutes, no rush)
3.                         result saved in DynamoDB / S3
4. Browser polls "is job 123 done?"  or is notified (WebSocket / SNS)
```

- The queue also **absorbs traffic spikes** and allows retries; messages that keep failing go to a **dead-letter queue (DLQ)** so nothing is silently lost.
- Bedrock's native async call **StartAsyncInvoke** handles slow generation types such as **Nova Reel video**; results are written to S3.

2.4 Quotas, throttling and retries

Each account has per-model, per-Region quotas: **TPM** (tokens per minute) and **RPM** (requests per minute). Exceeding them returns **ThrottlingException** (HTTP 429). Retry with **exponential backoff** (wait 1 s, 2 s, 4 s...) plus **jitter** (a little randomness so clients don't retry in sync). The AWS SDKs (e.g. boto3) do this already; you configure them.

```
Attempt 1 x (throttled) -> wait ~1s -> Attempt 2 x -> wait ~2s -> Attempt 3 x -> wait ~4s -> Attempt 4 OK
```

Error	Meaning	Retry?
ThrottlingException	Over quota right now	Yes, with backoff
ServiceUnavailableException / ModelNotReadyException	Temporary service-side issue	Yes, with backoff
ModelTimeoutException	Model took too long	Sometimes; consider shorter tasks or async
ValidationException	Request invalid (e.g. input exceeds context window)	**No** — fix the request
AccessDeniedException	IAM permissions missing	**No** — fix permissions

For an input that is too long: reduce it — send only relevant chunks with **RAG**, chunk or summarise, or use a model with a larger context window.

Example — Black Friday 10x traffic, cheapest fixes first: retries with backoff → cross-region inference → an SQS buffer (if slight delay is fine) → quota increase or Provisioned Throughput.

2.5 Cross-region inference

You call an **inference profile** instead of a single-Region model; Bedrock routes each request to a Region in that group with spare capacity.

```
Your app (us-east-1) -- modelId = "us.amazon.nova-lite-v1:0" --> Bedrock router
                                                                  |-> us-east-1
                                                                  |-> us-east-2  (whichever has capacity)
                                                                  |-> us-west-2
```

- Prefixes: **us.**, **eu.**, **apac.**, or **global.** (worldwide).
- More throughput, fewer throttles, **no extra charge**, only the model ID changes.
- Data residency: a **geographic** profile (eu.) keeps processing in that geography; avoid **global.** when data must stay in, say, the EU.
- **Application inference profiles**: your own tagged profiles to **track cost and usage per team**.

	Cross-region inference	Quota increase
Capacity from	Other Regions in the geography	More capacity in your one Region
Effort	Change model ID — instant, no approval	Service Quotas / Support request — needs approval
Cost	No extra charge	No extra charge (pay for tokens used)

Part 3 — Deployment and paying for models (Task 2.2)

```
                 +------------------- Amazon Bedrock ------------------+   +-- Amazon SageMaker AI --+
real-time, pay   | 1. On-demand            (shared capacity)           |   | 4. Your own endpoint    |
per token        | 2. Provisioned Throughput (reserved capacity)       |   |    (choose GPU, full    |
not real-time,   | 3. Batch inference (file in -> file out, ~50% less) |   |    control, more work)  |
cheapest         +-----------------------------------------------------+   +-------------------------+
```

3.1 On-demand

Pay per input/output token, no commitment, shared capacity, limited by quotas. Default for most apps and unpredictable traffic. Example: an internal HR chatbot used a few hundred times a day.

3.2 Provisioned Throughput

Buy **model units (MUs)** — dedicated slices of capacity — billed **per hour whether used or not**, optionally with 1- or 6-month commitments. For steady, high, predictable volume where throttling is unacceptable (e.g. a fraud-explanation service at a constant 2,000 requests/minute). Wrong choice for low or spiky traffic: you pay for idle hours. Classic exam answer for **serving a fine-tuned Bedrock model** (AWS now also offers on-demand serving for some custom models).

3.3 Batch inference

Put many prompts in a **JSONL** file (one JSON request per line) in S3, start a batch job, get results in S3, usually within hours, at **~50% of on-demand price**. For large non-urgent jobs (Practice Set Q22).

```
S3: input.jsonl (2M prompts) --> Bedrock batch job --> (hours later) S3: output.jsonl
```

	Batch inference	Async SQS pattern
What	Managed Bedrock job: one file in, one file out	Your architecture: queue + worker + store
Typical use	Millions of prompts nobody is waiting for	One user's long request, or buffering spikes
Results	Hours	Seconds to minutes
Price	~50% cheaper	Normal on-demand

3.4 Customisation ladder — cheapest that works

```
effort & cost
    ^   5. Continued pre-training  (domain LANGUAGE from unlabeled text)
    |   4. Fine-tuning             (TASK / STYLE from labeled examples)
    |   3. Distillation            (SMALL model behaves like a BIG one)
    |   2. RAG                     (the right FACTS at question time)   <- Day 1
    |   1. Prompt engineering      (better instructions + examples)     <- Day 7
    +-------------------------------------------------------------------->
```

- **Distillation**: a large **teacher** answers your prompts; those answers train a small **student**. Result: near-teacher quality for your use case at much lower cost/latency (support bot at 1M chats/month).
- **Fine-tuning**: hundreds–thousands of **labelled pairs** (prompt → ideal answer) change behaviour — format, tone, a specific task (strict JSON report format in brand voice).
- **Continued pre-training**: lots of **unlabelled domain text** teaches vocabulary (50,000 pharma research papers).
- Labelled pairs → fine-tuning; raw text → continued pre-training. Facts that change → RAG, not fine-tuning.
- Training data in S3; jobs can run in your VPC and be encrypted with your KMS key (Day 4).

3.5 Bringing your own model; SageMaker AI

- **Custom Model Import**: import weights of an open-weight model you trained elsewhere (e.g. Llama fine-tuned in SageMaker) from S3 and call it through the same Bedrock APIs, serverlessly.
- **Bedrock Marketplace**: extra specialised models on managed endpoints, called via Bedrock APIs.
- **SageMaker AI endpoints** (incl. **JumpStart** catalogue): host any model on a GPU instance you choose; full control, more operations. Use only when Bedrock can't do it.

3.6 Which deployment option?

Scenario	Choose
Typical app, unpredictable traffic, pay for use	On-demand
Steady high volume, guaranteed throughput	Provisioned Throughput
Serve a fine-tuned Bedrock model (classic answer)	Provisioned Throughput
Millions of prompts, no real-time need, lowest cost	Batch inference
Small fast model with big-model quality for one use case	Model distillation
Consistent format/style from labelled examples	Fine-tuning
Domain vocabulary from unlabelled documents	Continued pre-training
Up-to-date or private facts	RAG
Model trained elsewhere, serve via Bedrock serverlessly	Custom Model Import
Model not on Bedrock, specific GPU/container	SageMaker AI endpoint

Decision rules for Day 5

1. Most cost-effective → smallest model that's good enough, proven with **Bedrock model evaluation**.
2. Image/video/document input → multimodal; search vectors → embedding model; long input → large context window.
3. Consistency → low temperature; correctness → grounding; truncation (max_tokens) → raise max tokens.
4. Switch/compare models with minimal code → **Converse API**.
5. Feels slow → **streaming** (TTFT, not cost).
6. Long generations time out → **SQS + worker + stored result**; video → **StartAsyncInvoke**.
7. Throttling → backoff + jitter; ValidationException / AccessDenied → don't retry, fix.
8. More throughput, no extra cost → **cross-region inference profile** (geographic for residency).
9. Cost per team → **application inference profiles** with tags.
10. Steady guaranteed capacity → **Provisioned Throughput**; low/spiky → on-demand.
11. Huge non-urgent volume → **batch inference** (~50% cheaper).
12. Labelled pairs → fine-tuning; raw text → continued pre-training; small-like-big → distillation; trained elsewhere → Custom Model Import.

Your part checks — graded

Part	Your answers	Feedback
1	Model choice depends on use case; temperature vs top-p; max_tokens meaning	✓ / ½ / ½ — top-p is a **probability threshold**, not a count (that's top-k); at max_tokens the model **reaches** the limit and is stopped.
2	Streaming vs cost; ValidationException; cross-region vs quota	✓ ✓ ✓ — add RAG/summarising as fixes for too-long input; CRIS is instant with no approval.
3	PT for spiky traffic; fine-tuning vs continued pre-training; batch vs SQS	✓ ✓ ½ — the batch vs SQS difference is purpose, speed and price (table in 3.3).

Lab — console only, no model quota needed (status: to do later)

Part	Steps	Record (send to the professor)
A. Model catalogue (10 min)	Bedrock → Model catalog; filter by provider and modality; open Nova Micro, Lite, Pro and note input modalities and context window.	Pick a model for: (a) 5M short reviews, cheapest; (b) damage in parcel photos; (c) KB search vectors — with reasons.
B. Quotas (10 min)	Service Quotas → Amazon Bedrock: on-demand TPM / RPM for Nova Lite, batch quotas, Provisioned Throughput model-unit quota. **Request a TPM increase** for Nova Lite or Micro.	Current TPM / RPM; which quota relates to Provisioned Throughput.
C. Cross-region inference (10 min)	Bedrock → Cross-region inference: open the Nova Lite us./global. profiles; create an application inference profile in CloudShell (commands below), list it, delete it.	Regions in the us. profile; how a tagged profile splits costs by team.
D. Provisioned Throughput (5 min)	Purchase form: model units, commitment options. **Do not buy — Cancel.**	One scenario where it's right, one where it's wrong.
E. Batch inference (10 min)	Batch inference → Create job: input S3, output S3, service role. **Don't submit.**	Why every JSONL line has a recordId.
F. Customisation forms (10 min)	Custom models → Customize model: fine-tuning, continued pre-training, distillation; hyperparameters (epochs, learning rate, batch size), S3 data, VPC, KMS. Imported models → Import model. Cancel both.	Which form needs labelled pairs, which needs raw text.
G. Model evaluation (5 min)	Evaluations → Create: automatic, LLM-as-a-judge, human. Cancel.	Which type for 3 models × 200 open-ended support answers.

```
aws bedrock create-inference-profile \
  --inference-profile-name support-bot \
  --model-source copyFrom=<the us. Nova Lite inference profile ARN> \
  --tags key=team,value=support
aws bedrock list-inference-profiles --type-equals APPLICATION
aws bedrock delete-inference-profile --inference-profile-identifier <ARN from the create output>
```

Batch input format (one request per line; modelInput uses the model's request format):

```
{"recordId": "REV0000001", "modelInput": {"messages": [{"role": "user", "content": [{"text": "Classify senti
{"recordId": "REV0000002", "modelInput": {"messages": [{"role": "user", "content": [{"text": "Classify senti
```

Cleanup
Delete the application inference profile. Nothing else is created. Never press Purchase on Provisioned Throughput.

Day 5 quiz — 8 / 8

#	Scenario	Answer	Why
1	4M reviews/month, weekly dashboard, small model is accurate enough, lowest cost	B ✓	Batch inference ~50% cheaper; no real-time need. PT would pay for idle hours; SQS uses on-demand prices.
2	Different answers each time + quotes non-existent terms	A, C ✓	Lower temperature fixes inconsistency; grounding (KB + contextual grounding check) fixes made-up facts.
3	5-page reports take 2 minutes and time out; users will wait	C ✓	Async SQS + worker + stored result. Streaming doesn't remove the gateway timeout for the total duration.
4	Throttling at peaks; more throughput now, no cost, minimal change; data may be anywhere in US	B ✓	us. cross-region profile. global. could leave the US; PT costs money; moving Regions is a big change.
5	ValidationException: input too long; retries keep failing	C ✓	Don't retry validation errors; reduce input (RAG, summarise).
6	Great large-model answers at 3M chats/month; want small model, minimal labelling	B ✓	Distillation: teacher generates the training data, no manual labelling.
7	Llama fine-tuned in SageMaker; call via Bedrock APIs, no servers	B ✓	Custom Model Import from S3.
8	Compare 3 providers, switch cheaply later, track cost per team	A, C ✓	Converse API (one-line switch) + application inference profiles tagged per team.

Mistakes and learnings

Point	Correction
Top-p described as 'top p tokens'	p is a cumulative probability threshold; a fixed count is top-k.
max_tokens 'generated more than the limit'	It reaches the limit and is stopped there.
Batch vs SQS described as 'batches vs one at a time'	Difference is purpose, result speed and price.
Hallucination vs temperature	Temperature → consistency; grounding → correctness.

Glossary — new terms today

Term	Meaning
Foundation model (FM)	Large pre-trained general-purpose model you build on without training it yourself.
Modality / multimodal	Type of data a model handles; multimodal models accept e.g. text and images.
Context window	Maximum tokens a model can consider in one request.
Inference	Running a model to get an output.
Temperature	Randomness of token choice: low = consistent, high = creative.
Top-p (nucleus sampling)	Sample only from top tokens whose probabilities add up to p.
Top-k	Sample only from the k most likely tokens.
Max tokens / stop reason max_tokens	Answer length limit; stop reason shown when the limit is reached.
Stop sequence	String that ends generation when produced.
Bedrock model evaluation	Managed comparison of models on your dataset (automatic, LLM-as-a-judge, human).
InvokeModel	Model-specific Bedrock call; request format differs per provider.
Converse / ConverseStream	Unified Bedrock call format across models; streaming variant.
inferenceConfig / toolConfig	Converse fields for inference parameters / tool definitions.
Streaming / TTFT	Sending tokens as generated; time to first token = perceived latency.
Synchronous / asynchronous	Caller waits for result / result delivered later.
API Gateway timeout	Default 29-second integration timeout for synchronous APIs.
SQS / DLQ	Simple Queue Service; dead-letter queue for messages that keep failing.
StartAsyncInvoke	Bedrock native async call for slow generation (e.g. Nova Reel video) → S3.
TPM / RPM	Tokens / requests per minute quotas.
ThrottlingException (HTTP 429)	Over quota; retry with backoff.
Exponential backoff + jitter	Retries with growing, slightly randomised waits.
ValidationException	Invalid request (e.g. input too long); don't retry, fix it.
Cross-region inference / inference profile	Routing requests across Regions in a geography via a profile ID (us., eu., apac., global.).
Application inference profile	Your own tagged profile to track cost/usage per team.
On-demand	Pay per token, shared capacity, no commitment.
Provisioned Throughput / model unit (MU)	Reserved dedicated capacity billed hourly; MU = unit of capacity.
Batch inference / JSONL	File-based bulk job (~50% cheaper); JSON Lines = one JSON object per line.
Fine-tuning	Further training on labelled prompt→answer pairs to change behaviour.
Continued pre-training	Further training on unlabelled domain text to learn vocabulary.
Model distillation (teacher/student)	Large model's answers train a smaller model.
Hyperparameters (epochs, learning rate, batch size)	Training settings for customisation jobs.

Term	Meaning
Custom Model Import	Bring open-weight model weights from S3 into Bedrock, served serverlessly.
Bedrock Marketplace	Additional specialised models on managed endpoints via Bedrock APIs.
SageMaker AI / JumpStart	AWS ML platform; JumpStart = its pre-trained model catalogue; host on chosen instances.

Next: Day 6 — Guardrails and Responsible AI (Tasks 3.1, 3.4)
Content filters, denied topics, word filters, PII filters, contextual grounding, Automated Reasoning checks, prompt injection and jailbreaks, and responsible AI principles.